Proposta de Trabalho Final para disciplina Programação para Web II - 2026/2

Docente: Rodrigo Prestes Machado
Discente: Maycke Young de Lima

Conceito do sistema

O sistema consiste em uma aplicação distribuída voltada para o gerenciamento de missões intergalácticas e alocação de tripulantes. O objetivo principal é automatizar o controle de missões espaciais e garantir a elegibilidade da tripulação em tempo real, validando o status dos personagens por meio de uma fonte de dados externa. O sistema atende operadores de missões que precisam cadastrar novas explorações espaciais e assegurar que nenhum membro da tripulação esteja morto (status != "Alive") antes de autorizar a partida, utilizando a API pública de *Rick and Morty*, especificamente o endpoint “/character”, como fonte externa dos dados que servirão como base de validação.

Arquitetura de microsserviços

A arquitetura é composta por três microsserviços independentes desenvolvidos em Java com Quarkus, que se comunicam por meio de REST e REST Client.

- Auth-Service: Responsável por gerenciar a autenticação e a emissão de credenciais de acesso. Expõe endpoint REST para autenticação de usuários e geração do token JWT, garantindo que apenas clientes autenticados possam acessar os recursos protegidos dos demais microsserviços.
- Missao-Service: Responsável por gerenciar o ciclo de vida das missões. Expõe endpoints REST para criação, exclusão, listagem, busca por ID de missões e inclusão de tripulantes. Atua como o orquestrador do fluxo de negócio.
- Status-Personagem-Service: Responsável por atuar como intermediário e camada de integração com a API pública de *Rick and Morty*. Consulta os dados do personagem externo, filtra o atributo de status e retorna se o personagem está apto ("Alive") para a missão.

A comunicação entre os serviços ocorrerá da seguinte maneira:

Cliente → Auth-Service
Cliente → Missao-Service → Status-Personagem-Service → Rick and Morty API

Diagrama de Componentes

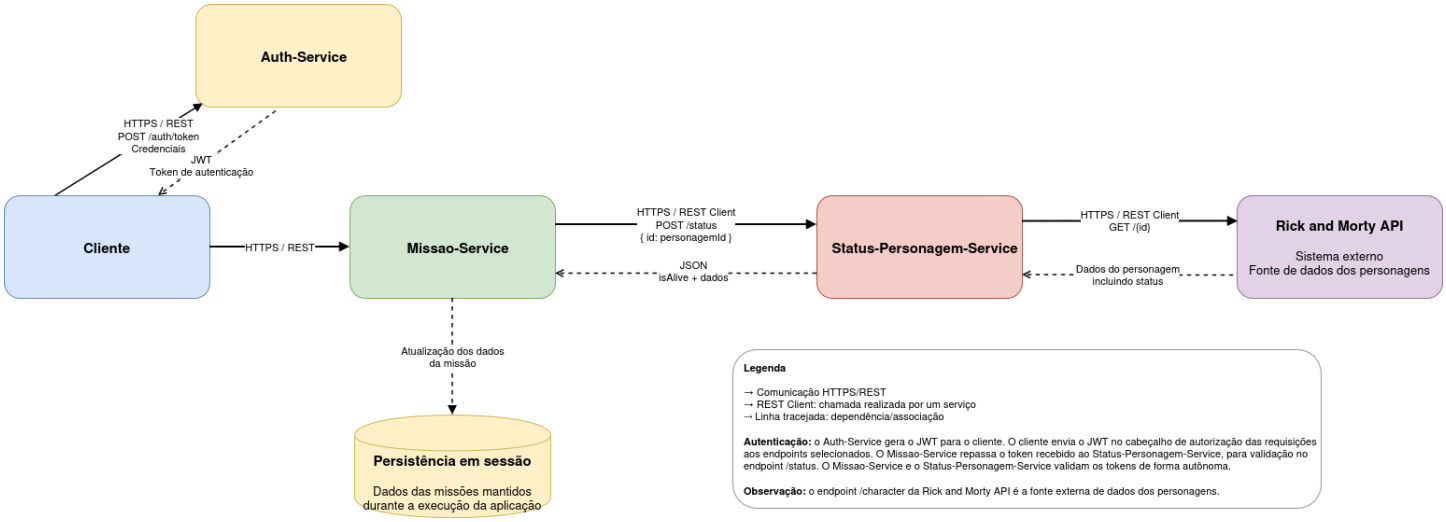
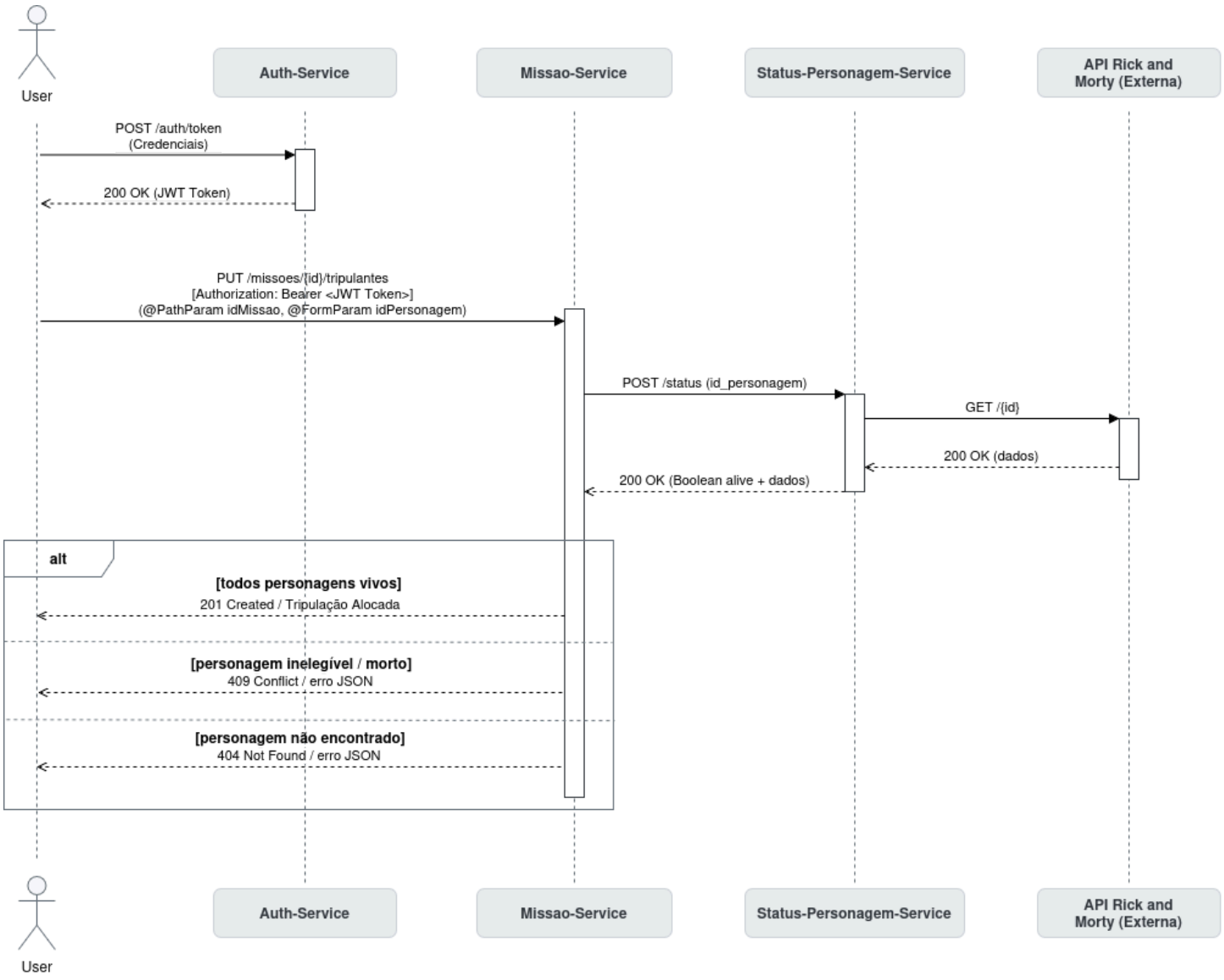


Diagrama de Sequência da funcionalidade disponível no endpoint “/atualizar-tripulantes”



Modelo de dados

No microsserviço Status-Personagem-Service, serão utilizados 2 modelos:

- Personagem = construído baseado no Character schema da API utilizada como fonte de dados externos.
- PersonagemResponse = um DTO (*Data Transfer Object*) composto pelos dados de um personagem mais um indicador booleano de vida (isAlive) desse personagem. O microsserviço avalia um atributo do personagem e salva o resultado de sua avaliação nesse indicador booleano. Esse DTO será repassado para o microsserviço Missao-Service.

No microsserviço Missao-Service será utilizado um outro modelo, além dos 2 modelos apresentados acima, totalizando 3 modelos. O modelo extra seria:

- Missao = contém os dados de uma missão registrada pelo serviço; cada missão é composta por:
    - ID: gerado automaticamente pelo serviço;
    - título: fornecido pelo cliente no momento de cadastro de uma nova missão;
    - descrição: fornecida pelo cliente no momento de cadastro de uma nova missão;
    - conjunto de personagens: fornecido pelo cliente em um segundo momento (após a criação da missão), mas somente registrado na entidade após validação unitária de cada personagem pelo microsserviço Status-Personagem-Service.

Lista de funcionalidades a serem desenvolvidas

Auth-Service: Autenticação do usuário/cliente. Endpoint:

1. POST “/auth/token” = Recebe credenciais via *URL-encoded form* e fornece token para acesso autenticado em endpoints específicos do sistema.

Missao-Service: Cria, lista ou exclui missões intergalácticas e atribui participantes. Constituído pelos endpoints:

2. GET “/missoes” = lista todas as missões cadastradas;
3. GET “/missoes?id={id}” = consulta uma missão específica, com o ID fornecido via *query*;
4. POST "/missoes" = Adiciona nova missão (título e descrição recebidos via JSON) [requer autenticação por token JWT];
5. DELETE "/missoes/{id}" = Exclui missão com ID fornecido via *path* [requer autenticação por token JWT];
6. PUT "/missoes/{id}/tripulantes" = Recebe ID da missão via *path* e um *URL-encoded form* com uma lista de IDs de personagens (possíveis tripulantes), verifica se missão existe, se sim, verifica se cada personagem existe, se sim, retorna JSON com a missão atualizada com tripulação/participantes [requer autenticação por token JWT].

Status-Personagem-Service: Consulta API externa em busca dos dados do possível tripulante, valida a elegibilidade do tripulante para participar na missão e retorna o resultado. Endpoint:

7. POST "/status" = recebe JSON com id de personagem do Missao-Service, consulta API externa para recuperar dados do personagem, e retorna JSON com um DTO composto pelos dados do personagem e um Boolean dizendo se está vivo ou não [requer autenticação por token JWT].

Requisitos não funcionais

JWT: autenticação por token, especificamente para acesso aos endpoints 4, 5 e 6 do serviço Missao-Service, de modo que um usuário sem autenticação consiga fazer consultas gerais, mas somente um

usuário autenticado consiga criar missões, excluir missões e alocar tripulantes para as missões. O serviço Status-Personagem-Service também requer esse mesmo token para autenticação no endpoint 7, que o recebe por repasse (REST Client) do serviço Missao-Service.

HTTPS: em toda comunicação envolvida, inclusive entre os serviços implementados, de modo a garantir a confidencialidade de toda comunicação via criptografia.

Referências

FUHRMANN, Axel. **The Rick and Morty API**: Documentation. [*S.l.*], 2026. Disponível em: https://rickandmortyapi.com/documentation. Acesso em: 16 set. 2026. Documentação da API externa utilizada.

MACHADO, Rodrigo Prestes. **HTTP**: simulador interativo do protocolo HTTP. Porto Alegre, 2026. Disponível em: https://cpw2.rpmhub.dev/http/. Acesso em: 16 set. 2026. Material utilizado para consulta aos códigos de status HTTP.

MACHADO, Rodrigo Prestes. **Programação para Web II (PW2)**. Porto Alegre, 2026. Disponível em: https://pw2.rpmhub.dev/. Acesso em: 16 set. 2026. Material de aula.

Comentários pessoais do autor sobre a ideia inicial do projeto

Meu objetivo inicial era fazer com que os serviços desenvolvidos para este trabalho tivessem alguma conexão com uma API externa, de modo que, além dos requisitos do trabalho, eu pudesse aprender também a conectar um serviço que eu desenvolvi com um serviço desenvolvido por um terceiro. Pensei inicialmente em buscar alguma API que fosse relacionada à previsão climática, mas durante as buscas me deparei com essa API relacionada a uma série da qual eu gosto, e assim surgiu a ideia de desenvolver algo que me permitisse contribuir para o “gerenciamento de missões intergaláticas”.